

So, what is this thing now?

Same idea as before, just leveled up. You're not limited to PDFs anymore — drop in a PDF, an audio file, or a video, and chat with it like you're talking to a friend who already went through the whole thing. It's basically becoming a one-stop "media intelligence" hub instead of just a document reader.

Still the same RAG + LLM engine under the hood, just taught to understand more than text now — it can listen to audio, watch video frames, and pull all of that into the same searchable brain.

The Stack (aka what it's built with)

Part	What we're using
Frontend	Streamlit — now with embedded video/audio players
Backend	FastAPI — handles the heavier async upload + processing jobs
Brains (LLM)	OpenAI GPT-4o or Gemini — upgraded to multimodal models since plain text LLMs can't read
Glue	LangChain — ties retrieval, multi-format chunking, and the LLM together
Embeddings	Sentence Transformers
Vector Store	FAISS or ChromaDB — now stores text, transcripts, and frame descriptions together
OCR	Tesseract (for scanned PDFs)
PDF Parsing	PyMuPDF, pdfplumber, PyPDF2
NLP bits	spaCy + NLTK
Speech (old)	SpeechRecognition / Whisper (STT), pyttsx3 / gTTS (TTS) — for typing-by-voice + read-aloud
Audio Processing (new)	OpenAI Whisper + MoviePy — rips audio out of files and transcribes it
Video Processing (new)	OpenCV (cv2) / Decord — samples video frames to understand what's on screen

Basically all Python, all open-source-friendly, nothing too exotic.

New Stuff — Audio & Video Features

This is the actual glow-up. Same "explain it like a friend" format — what it does, how it works, and where it's gonna trip up. These are the features that turn it from a PDF tool into an everything tool.

■ Video Chat & Analysis

Drop in an MP4/MKV — a lecture, a recorded meeting, whatever — and ask about both what was said *and* what was on screen.

■ ■ How it works

MoviePy rips the audio out for transcription, while OpenCV samples frames every second or so. Those frames get described by a vision-language model (or sent straight to a multimodal LLM), and both the transcript and the visual descriptions land in the same vector database.

■ ■ Where it doesn't work / falls short

Long, high-res videos eat a ton of processing time. Anything that flashes by quickly — tiny text on a background screen, fast cuts — can get missed since it's only sampling frames, not watching every second.

■ ■ Audio Chat & Transcription

Upload an MP3, WAV, or M4A — a voice note, a podcast, a lecture recording — and chat with it directly.

■ ■ How it works

The file goes straight into OpenAI Whisper (locally or via API), which spits out a solid, time-stamped transcript. That transcript gets chunked and stored exactly like the old PDF pipeline.

■ ■ Where it doesn't work / falls short

Multiple people talking over each other, heavy background noise, or dense jargon can all mess with transcript accuracy.

■ ■ Timestamped Referencing

When it answers from a video or audio file, it tells you exactly when that moment happened — like [02:14 - 02:45].

■ ■ How it works

During chunking, text segments get mapped to their timestamps. When RAG pulls the best-matching chunks for your question, it grabs those timestamps too and turns them into clickable references in the chat.

■ ■ Where it doesn't work / falls short

If something's discussed vaguely across a whole hour instead of one clear moment, the timestamp it points to might be a bit generic instead of the exact "aha" moment.

■ ■ Multimodal Preview Panel

A split-screen setup — watch the video or listen to the audio right next to the chat, instead of hunting for the file elsewhere.

■ ■ How it works

Streamlit's built-in media players embed the file directly. In a more advanced version, the timestamp data can auto-jump the player to the exact second an answer is talking about.

■ ■ Where it doesn't work / falls short

Big video files can lag or buffer slowly in-browser if the server isn't set up to stream media properly.

How the Old Pieces Evolve

Document Chunking → **Multimodal Chunking**: instead of just splitting by character count, chunks now split by paragraph (PDFs), time interval (audio), and scene/time block (video).

RAG Pipeline: a question now searches across every format at once. Ask "what did they say about the budget?" and it checks the PDF text, the audio transcript, and the video's visual descriptions all in the same search.

Original Features — the real breakdown

Here's every feature from the original PDF chatbot, explained like I'd explain it to a friend — what it actually does, how it works behind the scenes, and (important) where it kinda falls apart or just doesn't do certain things. All of this still applies — it's just the foundation the new media features sit on top of. ■

■ PDF Upload

You drop a PDF in and it's ready to chat with. That's it, that's the feature.

■■ How it works

File gets checked (right type, not too huge) and saved temporarily on the server so the pipeline can grab it.

■■ Where it doesn't work / falls short

Doesn't support Word docs, PowerPoints, or random file types — PDF only for now. Also huge files (think 500+ pages) will be slow.

■■ PDF Preview

Lets you actually look at the PDF on screen while you're chatting, side by side.

■■ How it works

The browser renders the PDF pages directly using a lightweight viewer — no separate app needed.

■■ Where it doesn't work / falls short

It's just a viewer — it won't highlight or jump to the exact page an answer came from (yet). You still have to scroll to double check.

■ OCR Support

Handles scanned PDFs — like an old photocopied document or a picture-turned-PDF.

■■ How it works

Each page gets turned into an image, then Tesseract OCR reads the text off it like a human squinting at a photo.

■■ Where it doesn't work / falls short

OCR isn't perfect — messy handwriting, low-quality scans, or weird fonts can trip it up and give garbled text.

■ Text Extraction

Pulls the actual readable text out of a normal (non-scanned) PDF.

■ ■ How it works

PyMuPDF and pdfplumber dig through the PDF's internal structure to get clean text in the right order.

■ ■ Where it doesn't work / falls short

Complex layouts — multi-column academic papers, weird tables, PDFs with text-as-images — can come out jumbled.

>■ Document Chunking

Breaks the document into smaller bite-sized pieces instead of feeding the whole thing in one go.

■ ■ How it works

Text gets split into overlapping chunks (like sliding a window across the doc) so nothing important gets cut off awkwardly.

■ ■ Where it doesn't work / falls short

Sometimes context that spans multiple chunks doesn't get retrieved together, so an answer might miss part of the picture.

■ Embedding Generation

Turns each chunk of text into a bunch of numbers that represent its *meaning*.

■ ■ How it works

A Sentence Transformers model reads the chunk and spits out a vector — similar meaning = similar numbers.

■ ■ Where it doesn't work / falls short

It's meaning-based, not fact-based — it can find similar-sounding text even if that text is actually wrong or irrelevant to the real answer.

■ ■ Vector Database Storage

Stores all those number-vectors somewhere so they can be searched fast later.

■ ■ How it works

FAISS or ChromaDB indexes everything so finding "closest matches" takes milliseconds, not minutes.

■ ■ Where it doesn't work / falls short

It's all in-memory / local by default — doesn't scale to enterprise-level, thousands-of-documents setups without extra work.

■ RAG Pipeline

The actual "find the relevant part of the doc, then answer" logic.

■ ■ How it works

Your question gets embedded too, matched against stored chunks, and the top matches get handed to the LLM as context.

■ ■ Where it doesn't work / falls short

If the retrieval step grabs the wrong chunks, the answer will be wrong too — garbage in, garbage out.

■ LLM-Based Question Answering

This is what actually writes the answer you read.

■ ■ How it works

The LLM (GPT or a local model) takes your question + the retrieved chunks and generates a proper, readable answer.

■ ■ Where it doesn't work / falls short

It can still occasionally hallucinate or misread context, especially with ambiguous or oddly-phrased questions. It's good, not infallible.

■ PDF Chat

The actual chat window where you type your questions.

■ ■ How it works

Every message runs through the RAG pipeline and shows up in a normal-looking chat thread.

■ ■ Where it doesn't work / falls short

It's single-document focused by default — chatting across multiple PDFs at once isn't there yet.

■ Document Summarization

Gives you the TL;DR of the whole document (or a section of it).

■ ■ How it works

Chunks get summarized individually, then those mini-summaries get combined into one final summary.

■ ■ Where it doesn't work / falls short

Really long or dense documents can lose some nuance — a summary is still a summary, not the full picture.

■ Topic Extraction

Tells you what the document is generally about, at a glance.

■ How it works

spaCy and NLTK dig through the text for recurring important terms and themes.

■ Where it doesn't work / falls short

Works best on text-heavy docs — sparse or highly technical documents can give vague or slightly-off topics.

■ Chat History

Keeps track of everything you asked so you can scroll back up.

■ How it works

Every question and answer gets logged with a timestamp for the current session.

■ Where it doesn't work / falls short

History resets or has to be manually saved — it's not automatically synced across devices or sessions unless that's set up separately.

■ Conversation Memory

Understands follow-up questions like "what about the second one?" without you repeating yourself.

■ How it works

Recent chat turns get included in the prompt sent to the LLM, so it has context of what you're referring to.

■ Where it doesn't work / falls short

Memory isn't infinite — go too many messages deep and earlier context can start getting dropped.

■ Export Summary

Lets you download the summary or the whole chat to keep for later.

■ How it works

Formats the text and saves it as a plain text or PDF file you can grab.

■ Where it doesn't work / falls short

Basic formatting only — don't expect a beautifully designed export doc, it's function over form.

■ Text-to-Speech (TTS)

Reads the answer out loud to you.

■ ■ How it works

The answer text goes through pyttsx3 (offline) or gTTS (better quality, needs internet) and plays as audio.

■ ■ Where it doesn't work / falls short

Offline voice (pyttsx3) sounds pretty robotic. The natural-sounding one (gTTS) needs an internet connection.

■ Speech-to-Text (STT)

Lets you just talk instead of typing your question.

■ ■ How it works

Your mic audio gets transcribed via SpeechRecognition or Whisper, then treated like a normal typed question.

■ ■ Where it doesn't work / falls short

Background noise or strong accents can mess with accuracy sometimes — it's good, not studio-perfect.

Real talk, before you go ■

This still isn't meant to replace actually reviewing important media carefully — contracts, legal recordings, anything with real stakes. Treat it as a fast way to get oriented and find the right spot, not the final word. Always double check anything that actually matters against the real file.

And now that logic applies to video and audio too — bad scan, bad audio, or a blurry video in means a shakier answer out. Garbage in, garbage out, just across more formats now.